МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни «Професійний Пайтон»

на тему: «Організація професійного Python-проєкту та Git»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Ознайомитися з принципами організації професійного Python-проєкту: навчитися створювати ізольоване віртуальне середовище, формувати структуру проєкту за принципом src-layout, розділяти програму на модулі й пакети, описувати метадані у файлі `pyproject.toml`, вести історію змін у Git та оформлювати базову документацію (README, .gitignore).

2. Індивідуальне завдання

Варіант 15. Облік результатів наукових експериментів. Для кожного експерименту зберігаються назва, дослідник (researcher), параметр, результат і дата. Необхідно реалізувати:

- додавання експерименту;
- пошук за researcher;
- обчислення середнього значення результатів;
- визначення мінімального та максимального результату;
- фільтрацію результатів за заданим діапазоном.

3. Постановка задачі

Потрібно створити невеликий консольний Python-застосунок для свого варіанта, який:

- має структуру src-layout і щонайменше три Python-модулі;
- містить data model (`Experiment`) та окремий модуль бізнес-логіки (`services.py`);
- має точку входу `main.py` з конструкцією `if __name__ == "__main__":`;
- використовує type hints, містить щонайменше три функції, приймає декілька вхідних значень, обробляє їх і формує зрозумілий результат;
- має файли `README.md`, `.gitignore`, `pyproject.toml` і зберігається в Git-репозиторії з кількома логічними комітами.

Проєкт отримав назву **experiment-registry** (пакет `experiment_registry`). Він стане основою наскрізного проєкту курсу: у наступних лабораторних до нього поступово додаватимуться структури даних, потокова обробка, ООП, тести, база даних, REST API, оптимізація та контейнеризація.

4. Хід виконання

4.1. Віртуальне середовище та встановлення пакета

У каталозі проєкту створено віртуальне середовище `.venv`, після активації якого пакет встановлено в editable-режимі (`pip install -e .`). Завдяки цьому зміни у вихідному коді одразу доступні без повторного встановлення, а в середовищі з’являється консольна команда `experiment-registry`, описана в `pyproject.toml`.

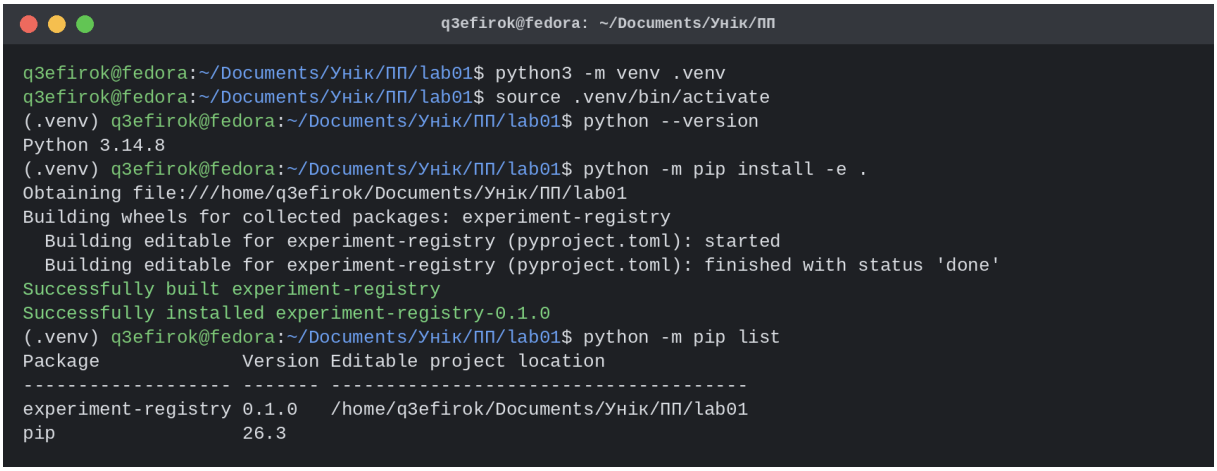


Рис. 1.1. Створення .venv, встановлення пакета в editable-режимі та перелік пакетів

4.2. Структура проєкту

Проєкт побудовано за принципом src-layout: увесь програмний код розміщено в `src/experiment_registry/`. Такий підхід не дає випадково імпортувати пакет із кореня репозиторію без встановлення і змушує перевіряти саме встановлену версію.

Лістинг 1.1. Структура проєкту lab01

```
lab01/
├── src/
│   └── experiment_registry/
│       ├── __init__.py
│       ├── config.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       └── services.py
├── .gitignore
├── pyproject.toml
└── README.md
```

4.3. Опис створених модулів

Таблиця 1.1. Модулі пакета experiment_registry

Модуль	Призначення
`__init__.py`	Позначає каталог як Python package, містить версію `__version__`.
`config.py`	Конфігураційні константи: формат дати, допустимий діапазон результату, ширина колонки таблиці.
`exceptions.py`	Власні винятки: `ExperimentError`, `ExperimentValidationError`, `DuplicateExperimentError`.
`models.py`	Data model `Experiment` (dataclass з валідацією у `__post_init__`) та `ResultStatistics`.
`services.py`	Бізнес-логіка: додавання, пошук, середнє, мінімум/максимум, фільтрація за діапазоном, сортування, пошук за кількома критеріями.
`main.py`	Точка входу: демонстраційний сценарій, консольне меню, введення даних із клавіатури з перевіркою.

4.4. Основні фрагменти коду

Модель експерименту реалізовано як `dataclass`, що автоматично генерує конструктор, `__repr__` та `__eq__`. Перевірка коректності даних виконується в `__post_init__`: неможливо створити експеримент із порожньою назвою чи результатом поза допустимим діапазоном. Властивість `label` формує короткий опис для виведення.

Лістинг 1.2. Файл models.py

```
"""Data models of the experiment registry."""

from dataclasses import dataclass
from datetime import date

from experiment_registry.config import MAX_RESULT, MIN_RESULT
from experiment_registry.exceptions import ExperimentValidationError


@dataclass
class Experiment:
    """Single scientific experiment and its numeric result."""

    experiment_id: int
    name: str
    researcher: str
    parameter: str
    result: float
    date: date

    def __post_init__(self) -> None:
        if self.experiment_id <= 0:
            raise ExperimentValidationError("Experiment ID must be positive.")
        if not self.name.strip():
            raise ExperimentValidationError("Experiment name must not be empty.")
        if not self.researcher.strip():
            raise ExperimentValidationError("Researcher must not be empty.")
        if not self.parameter.strip():
            raise ExperimentValidationError("Parameter must not be empty.")
        if not MIN_RESULT <= self.result <= MAX_RESULT:
            raise ExperimentValidationError(
                f"Result {self.result} is outside [{MIN_RESULT}, {MAX_RESULT}]."
            )

    @property
    def label(self) -> str:
        """Short human-readable description of the experiment."""
        return f"#{self.experiment_id} {self.name} ({self.researcher})"


@dataclass(frozen=True)
class ResultStatistics:
    """Aggregated statistics of experiment results."""

    count: int
    average: float
    minimum: float
    maximum: float
```

Модуль `services.py` містить функції бізнес-логіки. Вони не виконують введення-виведення й не залежать від консолі, тому їх можна повторно використати в API чи тестах.

Лістинг 1.3. Основні функції services.py

```
def add_experiment(experiments: list[Experiment], experiment: Experiment) -> None:
    """Add an experiment to the registry, rejecting duplicate IDs."""
    if any(item.experiment_id == experiment.experiment_id for item in experiments):
```

```
        raise DuplicateExperimentError(
            f"Experiment with ID {experiment.experiment_id} already exists."
        )
    experiments.append(experiment)


def find_by_researcher(experiments: list[Experiment], researcher: str) ->
list[Experiment]:
    """Return experiments of a researcher (case-insensitive match)."""
    wanted = researcher.strip().lower()
    return [item for item in experiments if item.researcher.lower() == wanted]


def calculate_average_result(experiments: list[Experiment]) -> float | None:
    """Return the mean result or None for an empty list."""
    if not experiments:
        return None
    return sum(item.result for item in experiments) / len(experiments)


def find_min_max(experiments: list[Experiment]) -> tuple[Experiment, Experiment] | None:
    """Return experiments with the minimum and the maximum result."""
    if not experiments:
        return None
    minimum = min(experiments, key=lambda item: item.result)
    maximum = max(experiments, key=lambda item: item.result)
    return minimum, maximum


def filter_by_result_range(
    experiments: list[Experiment],
    low: float,
    high: float,
) -> list[Experiment]:
    """Return experiments whose result lies in the closed range [low, high]."""
    if low > high:
        raise ExperimentValidationError(f"Invalid range: {low} > {high}.")
    return [item for item in experiments if low <= item.result <= high]
```

Лістинг 1.4. Пошук за кількома критеріями та сортування

```
def search(
    experiments: list[Experiment],
    researcher: str | None = None,
    parameter: str | None = None,
) -> list[Experiment]:
    """Search experiments by several optional criteria."""
    result = experiments
    if researcher:
        result = find_by_researcher(result, researcher)
    if parameter:
        result = filter_by_parameter(result, parameter)
    return result


def sort_experiments(
    experiments: list[Experiment],
    key: str = "date",
    reverse: bool = False,
) -> list[Experiment]:
    """Return experiments sorted by date, result or name."""
    if key not in {"date", "result", "name"}:
        raise ExperimentValidationError(f"Unknown sort key: {key}.")
    return sorted(experiments, key=lambda item: getattr(item, key), reverse=reverse)
```

Модуль `main.py` відокремлює запуск застосунку від бізнес-логіки. Функція `main()` виконується лише тоді, коли модуль запущено безпосередньо або через консольну команду.

Лістинг 1.5. Перевірка введення та точка входу main.py

```
def read_float(prompt: str) -> float:
    """Read a float from the keyboard until the value is valid."""
    while True:
        raw = input(prompt).strip().replace(",", ".")
        try:
            return float(raw)
        except ValueError:
            print("  Please enter a number.")


def main() -> None:
    """Run the demo scenario or the interactive menu (--menu)."""
    experiments = create_demo_experiments()
    if "--menu" in sys.argv[1:]:
        run_menu(experiments)
    else:
        run_demo(experiments)
```

Лістинг 1.6. Завершення модуля main.py

```
if __name__ == "__main__":
    main()
```

Лістинг 1.7. Файл exceptions.py

```
"""Custom exceptions of the experiment registry."""


class ExperimentError(Exception):
    """Base class for all errors of the application."""


class ExperimentValidationError(ExperimentError):
    """Raised when experiment data are invalid."""


class DuplicateExperimentError(ExperimentError):
    """Raised when an experiment with the same ID already exists."""
```

4.5. Файл pyproject.toml

Лістинг 1.8. Файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "experiment-registry"
version = "0.1.0"
description = "Registry of scientific experiment results (Professional Python, lab 1)"
readme = "README.md"
requires-python = ">=3.11"
authors = [{ name = "Volodymyr Nakladiuk" }]

[project.scripts]
experiment-registry = "experiment_registry.main:main"

[tool.setuptools.packages.find]
where = ["src"]
```

- `[build-system]` задає інструмент збирання (setuptools ≥ 70) і backend `setuptools.build_meta`;

- `[project]` описує метадані пакета: назву `experiment-registry`, версію 0.1.0, опис, README, мінімальну версію Python 3.11 та автора;
- `[project.scripts]` створює консольну команду `experiment-registry`, яка викликає функцію `experiment_registry.main:main`;
- `[tool.setuptools.packages.find]` вказує, що пакети шукаються в каталозі `src` (src-layout).

4.6. Файл .gitignore

Файл `.gitignore` виключає з репозиторію віртуальне середовище, кеші інтерпретатора, налаштування IDE, артефакти збирання та файл із секретами `.env`.

Лістинг 1.9. Файл .gitignore

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

4.7. README.md

Лістинг 1.10. Фрагмент README.md

````
# Experiment Registry

Навчальний проєкт з дисципліни «Професійний Пайтон».
Лабораторна робота №1 — організація професійного Python-проєкту та Git.

Варіант 15 — облік результатів наукових експериментів.
Для кожного експерименту зберігаються назва, дослідник (researcher), параметр, результат і
дата.

## Можливості

- додавання експерименту з перевіркою даних;
- пошук експериментів за дослідником;
- середнє значення, мінімум і максимум результатів;
- фільтрація результатів за заданим діапазоном;
- пошук за кількома критеріями та сортування;
- консольне меню з введенням даних із клавіатури.

## Вимоги

Python 3.11+

## Встановлення

```bash
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install -e .
```

## Запуск

Демонстраційний сценарій:

```bash
python -m experiment_registry.main
# або
````
````

````
experiment-registry
```

Інтерактивне меню:

```bash
python -m experiment_registry.main --menu
```

Приклад результату:
````

4.8. Запуск застосунку

Застосунок запускається як модуль (`python -m experiment_registry.main`) або через консольну команду. Демонстраційний сценарій виводить усі експерименти, додає новий, шукає експерименти дослідниці Olena Kovalenko, обчислює середнє, мінімум і максимум для параметра temperature, фільтрує результати в діапазоні [290, 320], виконує пошук за двома критеріями та сортування.

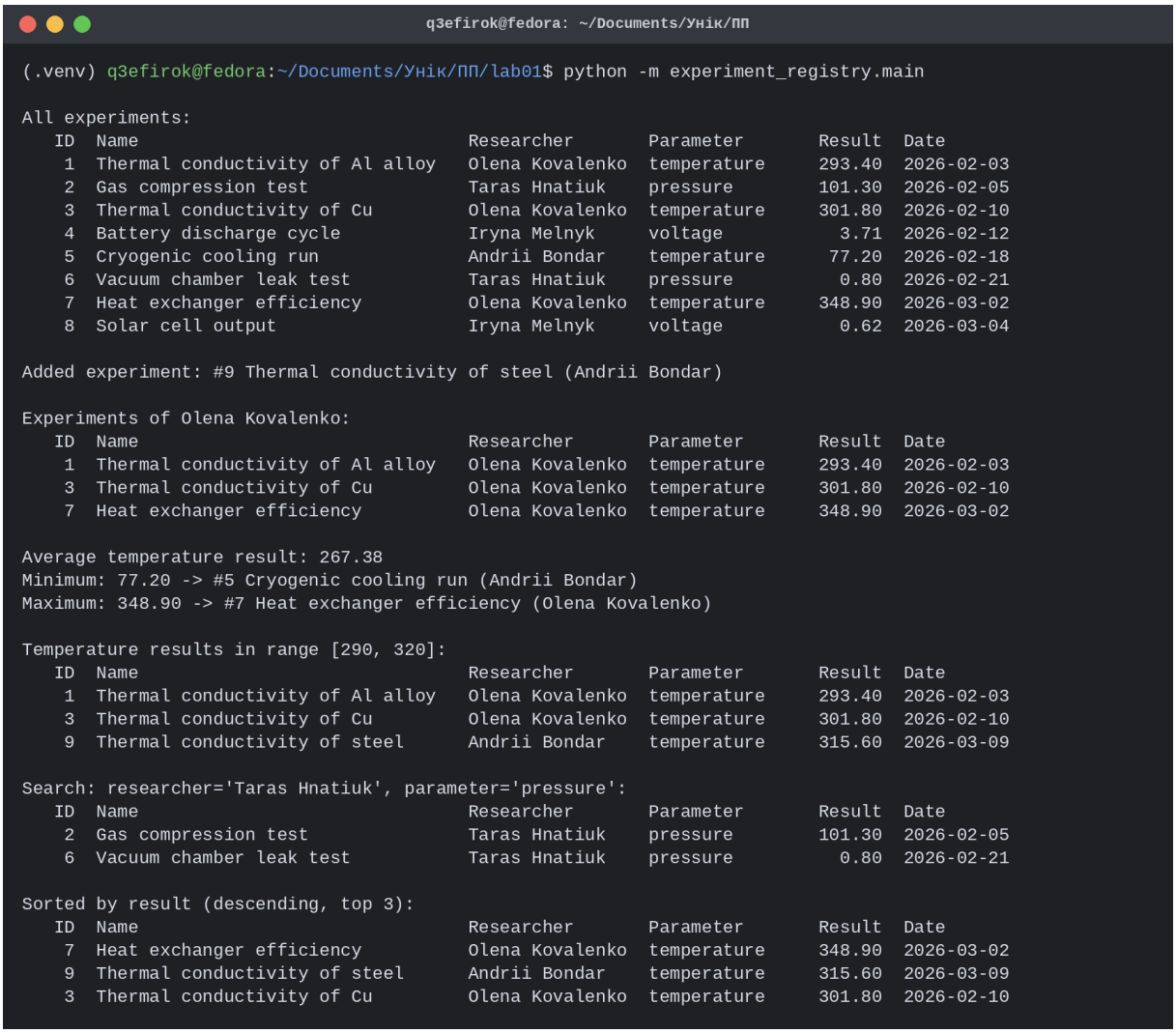


Рис. 1.2. Запуск демонстраційного сценарію як Python module
````



Рис. 1.3. Запуск через консольну команду experiment-registry

Інтерактивний режим (`--menu`) реалізує menu-driven interface. На знімку показано обчислення статистики для параметра, реакцію на некоректний діапазон (low > high), повторний запит числа після введення `abc` та додавання нового експерименту.

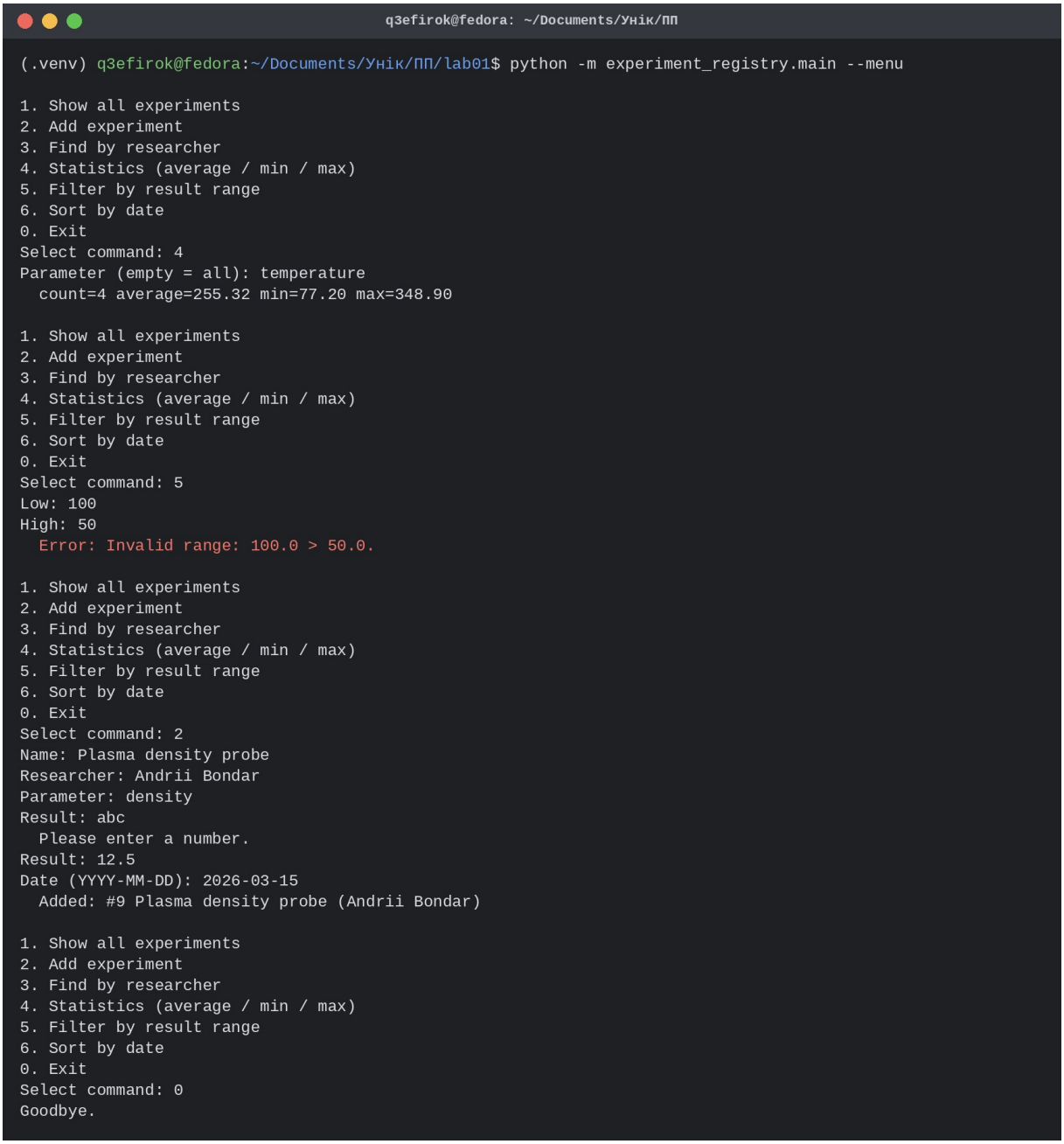


Рис. 1.4. Робота консольного меню з введенням даних та валідацією

4.9. Робота з Git

Репозиторій ініціалізовано командою `git init`, віддалений репозиторій підключено через `git remote add origin`. Розробку розбито на логічні коміти: створення структури, модель і бізнес-логіка, точка входу й документація.



Рис. 1.5. Результат git status

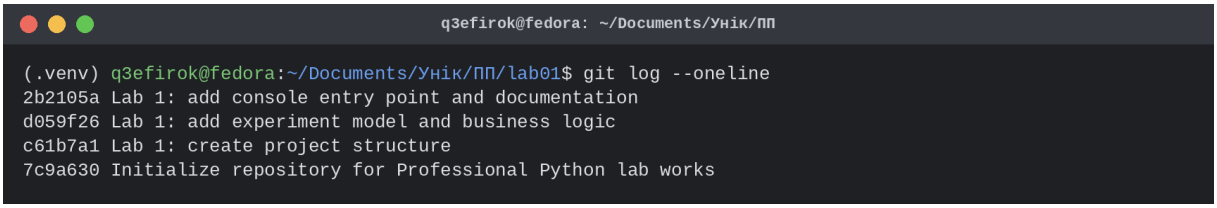


Рис. 1.6. Результат git log --oneline

4.10. Завдання підвищеної складності

- введення даних із клавіатури та menu-driven interface (`run_menu`);
- validation input: `read_float`, `read_date` повторюють запит до коректного значення, модель перевіряє поля в `__post_init__`;
- сортування за датою, результатом або назвою (`sort_experiments`);
- пошук за декількома критеріями (`search`);
- статистичні показники (`calculate_statistics`);
- окремі configuration constants (`config.py`);
- власні винятки (`exceptions.py`) та використання dataclass;
- README з прикладом запуску програми.

5. Відповіді на контрольні запитання

1. Що таке Python-проєкт?

Python-проєкт — це структурована сукупність пакетів і модулів з вихідним кодом, тестів, конфігурації (pyproject.toml), документації (README.md), правил для Git (.gitignore) та опису залежностей, яку можна відтворювано встановити й запустити в ізольованому середовищі. Наприклад, мій проєкт experiment-registry містить пакет src/experiment_registry з модулями models.py, services.py і main.py, а також pyproject.toml, README.md і .gitignore.

2. Чим проєкт відрізняється від одного Python script?

Script — це один .py файл, який просто запускається інтерпретатором і зручний для невеликих разових задач. Проєкт поділено на логічні модулі й пакети, він має метадані та залежності в pyproject.toml, ізольоване virtual environment, тести, документацію та історію змін у Git, тому його легше встановлювати, тестувати, супроводжувати й розвивати в команді.

3. Що таке Python module?

Python module — це окремий файл із розширенням .py, що містить визначення функцій, класів і змінних, має власний простір імен і може бути імпортований іншим кодом через import. Наприклад, models.py у моєму проєкті — модуль з dataclass Experiment, а services.py — модуль із функціями обробки експериментів; імпорт виглядає так: from experiment_registry.models import Experiment.

4. Що таке Python package?

Python package — це каталог, що об'єднує пов'язані модулі (і, можливо, підпакети) під спільним ім'ям та утворює ієрархічний простір імен; зазвичай він містить файл __init__.py. Модулі пакета імпортуються через крапку, наприклад import

experiment_registry.services. У моєму проєкті пакетом є каталог src/experiment_registry.

5. Для чого потрібний __init__.py?

Файл __init__.py позначає каталог як звичайний (regular) package і виконується під час першого імпорту пакета. У ньому можна розмістити docstring пакета, метадані на кшталт __version__ = "0.1.0", код ініціалізації або реекспорт публічних імен. Без нього Python 3.3+ трактує каталог як namespace package (PEP 420), але для явності й сумісності з інструментами пакування його рекомендують створювати.

6. Що таке src-layout?

src-layout — спосіб організації проєкту, за якого пакет із програмним кодом розміщують у підкаталозі src, а в корені залишаються лише конфігурація, документація та tests. Для мого проєкту це виглядає так: у корені pyproject.toml, README.md і .gitignore, а код — у src/experiment_registry/ (__init__.py, config.py, exceptions.py, models.py, services.py, main.py); каталог tests/ з’явиться в лабораторній №6.

7. Які переваги має src-layout?

src-layout не дозволяє випадково імпортувати пакет напряму з поточного робочого каталогу, тому програма й тести працюють саме з установленим пакетом (python -m pip install -e .), а помилки пакування виявляються одразу. Крім того, він чітко відокремлює вихідний код від тестів, конфігурації та службових файлів і запобігає конфліктам імен модулів із файлами в корені проєкту.

8. Для чого використовується virtual environment?

Virtual environment — ізольоване середовище Python з власним інтерпретатором (копією або посиланням), власним pip і окремим каталогом site-packages для бібліотек конкретного проєкту. Воно запобігає конфліктам версій пакетів між різними проєктами, не засмічує системний Python і дозволяє відтворити однакове середовище на іншому комп'ютері за описом залежностей.

9. Як створити .venv?

У кореневому каталозі проєкту потрібно виконати python -m venv .venv (у Windows також можна py -m venv .venv). Стандартний модуль venv створить каталог .venv з інтерпретатором, pip, скриптами активації та файлом pyvenv.cfg. Після цього каталог .venv/ обов'язково додають до .gitignore.

10. Як активувати virtual environment?

У Linux/macOS виконують source .venv/bin/activate, у Windows — .venv\Scripts\activate у cmd або .venv\Scripts\Activate.ps1 у PowerShell. Після активації команди python і pip вказують на інтерпретатор із .venv, а в запрошенні терміналу з'являється префікс (.venv); перевірити це можна через python --version або which python. Вийти із середовища можна командою deactivate.

11. Для чого використовується pip?

pip — стандартний менеджер пакетів Python, що встановлює, оновлює й видаляє пакети з PyPI або з локального каталогу та автоматично розв'язує їхні залежності.

Наприклад, python -m pip install -e . встановлює мій проєкт experiment-registry у .venv, python -m pip list показує встановлені пакети, а python -m pip freeze фіксує їхні версії.

12. Чому рекомендується python -m pip?

Команда python -m pip запускає pip як модуль саме того інтерпретатора, який викликано командою python, тож пакети гарантовано встановлюються в потрібне середовище, наприклад в активоване .venv. Окрема команда pip може належати іншій версії Python із PATH, і пакет потрапить не туди. Крім того, у Windows лише так можна коректно оновити сам pip: python -m pip install --upgrade pip.

13. Для чого потрібний pyproject.toml?

pyproject.toml — стандартний конфігураційний файл Python-проєкту у форматі TOML (PEP 518, PEP 621). Секція [build-system] задає інструмент збирання (наприклад, setuptools), [project] — метадані пакета (назва experiment-registry, версія, requires-python, dependencies), а секції [tool.*] — налаштування pytest, ruff, mypy тощо. Завдяки йому pip знає, як зібрати й установити проєкт.

14. Що містить секція [project]?

Секція [project] містить метадані пакета за PEP 621: обов'язкову назву name (у мене experiment-registry), version, description, requires-python (наприклад, ">=3.11"), а також за потреби readme, authors, license, dependencies (залежності часу виконання) й optional-dependencies. Пов'язана таблиця [project.scripts] описує console commands, які викликають функцію, наприклад main() з модуля experiment_registry.main.

15. Що таке editable installation?

Editable installation (python -m pip install -e .) — встановлення пакета в режимі розробки, коли в site-packages потрапляє не копія коду, а посилання на каталог проєкту (через .pth-файл або import hook). Тому зміни в src/experiment_registry одразу доступні без перевстановлення, а пакет імпортується за ім'ям з будь-якого місця. Перевстановлення потрібне лише після зміни метаданих, наприклад [project.scripts].

16. Для чого використовується if __name__ == "__main__"?

Змінна __name__ дорівнює "__main__", коли модуль запускається безпосередньо (наприклад, python -m experiment_registry.main), і дорівнює імені модуля, коли його імпортують. Тому код під цією умовою, зазвичай виклик main(), виконується лише під час прямого запуску, а при імпорті, наприклад у тестах, консольне меню не стартує автоматично.

17. Що таке Git?

Git — розподілена система контролю версій (distributed version control system), що зберігає історію змін файлів у вигляді commits, дозволяє повертатися до попередніх версій, працювати з branches, відстежувати авторство змін і співпрацювати в команді. Кожен розробник має повну локальну копію repository з усією історією, а синхронізація з GitHub чи GitLab виконується через push і pull.

18. Що таке repository?

Repository — сховище проєкту під контролем Git: робочі файли разом зі службовим каталогом .git, у якому зберігаються всі commits, branches, теги та конфігурація. Локальний repository створюється командою git init у корені проєкту, а віддалений (remote) розміщують на GitHub чи GitLab. Мій проєкт experiment-registry є таким repository з історією логічних commits.

19. Що таке commit?

Commit — зафіксований знімок (snapshot) стану файлів проєкту в певний момент, який має унікальний хеш SHA, автора, дату, повідомлення та посилання на батьківський commit. Послідовність commits утворює історію проєкту. Добрий commit містить одну логічну зміну: наприклад, окремими commits фіксують структуру проєкту, модель Experiment з business logic, а потім main.py і README.

20. Для чого використовується git status?

git status показує поточний стан робочого каталогу та staging area: активну branch, змінені файли, файли, уже додані до індексу для наступного commit (staged), і файли, які не відстежуються (untracked). Команда нічого не змінює, тому її виконують перед git add і git commit, щоб переконатися, що в commit не потраплять зайві файли, наприклад .venv чи __pycache__.

21. Для чого git add?

git add додає поточний вміст змінених або нових файлів до staging area (індексу), тобто формує вміст наступного commit. Можна додавати окремі файли, наприклад git add src/experiment_registry/models.py src/experiment_registry/services.py, щоб зробити логічно цілісний commit, або всі зміни командою git add . з урахуванням .gitignore.

22. Для чого git commit?

git commit створює новий commit зі змін, що перебувають у staging area, і записує його в історію repository разом з автором, датою та повідомленням. Повідомлення задають параметром -m, наприклад git commit -m "Add experiment model and business logic"; воно має коротко описувати одну логічну зміну.

23. Що показує git log?

git log показує історію commits поточної branch у зворотному хронологічному порядку: хеш, автора, дату й повідомлення кожного commit. Варіант git log --oneline виводить по одному рядку на commit (скорочений хеш і повідомлення), а --graph додатково малює розгалуження branches. Так можна продемонструвати, що проєкт має кілька логічних commits.

24. Для чого потрібен .gitignore?

.gitignore містить шаблони файлів і каталогів, які Git не повинен відстежувати й додавати до repository: virtual environment (.venv/), кеші (__pycache__/, *.pyc, .pytest_cache/), налаштування IDE (.idea/, .vscode/), build artifacts (dist/, build/, *.egg-info/) та секрети (.env). Це зберігає repository чистим і безпечним. На файли, які вже відстежуються, .gitignore не впливає.

25. Чому .venv не потрібно комітити?

.venv містить тисячі файлів установлених пакетів і шляхи, прив'язані до конкретної ОС, версії Python і розташування на диску, тому на іншому комп'ютері воно не працюватиме, а лише роздує repository. Середовище легко відтворити з опису залежностей у pyproject.toml командами python -m venv .venv та python -m pip install -e ., тому .venv/ додають до .gitignore.

26. Для чого потрібний README?

README.md — головний документ проєкту, який першим бачать у repository. Він описує призначення програми, вимоги (Python 3.11+), інструкції зі встановлення та запуску (python -m venv .venv, python -m pip install -e ., python -m experiment_registry.main), структуру модулів і автора. У моєму випадку README пояснює, що програма веде облік результатів наукових експериментів і які дії доступні в меню.

27. Що таке type hints?

Type hints — анотації типів параметрів, значень, що повертаються, і змінних (PEP 484), наприклад поле result: float у dataclass Experiment або функція пошуку із сигнатурою (experiments: list[Experiment], researcher: str) -> list[Experiment]. Інтерпретатор не перевіряє їх під час виконання, але вони документують код, покращують автодоповнення в IDE і дозволяють mypy чи pyright знаходити помилки типів до запуску.

28. Для чого програму розділяють на modules?

Поділ на modules реалізує принцип розділення відповідальностей: кожен модуль відповідає за одну частину системи, що полегшує читання, тестування, повторне використання й супровід коду, а також паралельну роботу кількох розробників. У моєму проєкті models.py містить модель даних Experiment, services.py — business logic (пошук, середнє, min/max, фільтр за діапазоном), а main.py — введення-виведення та консольне меню.

29. Що таке business logic?

Business logic — частина програми, що реалізує правила й обчислення предметної області незалежно від способу введення-виведення даних. У моєму варіанті це функції services.py: додавання експерименту, пошук за researcher, обчислення середнього результату, minimum/maximum і фільтрація результатів за заданим діапазоном. Вони приймають і повертають дані, не використовуючи input() чи print().

30. Чому потрібно відокремлювати application entry point від business logic?

Entry point (main.py з меню, input() і print()) відповідає лише за взаємодію з користувачем, а business logic у services.py — за обчислення. Таке розділення дозволяє тестувати логіку автоматично без консолі, повторно використовувати її з іншим інтерфейсом (GUI, web API, аргументи командного рядка) і змінювати меню, не зачіпаючи обчислень, і навпаки.

6. Висновок

У лабораторній роботі створено структурований Python-проєкт experiment-registry для обліку результатів наукових експериментів. Код розділено на модулі за відповідальністю: модель даних, бізнес-логіка, конфігурація, винятки та точка входу. Проєкт використовує src-layout, встановлюється у віртуальне середовище командою `pip install -e .` і надає консольну команду `experiment-registry`.

Реалізовано всі функції варіанта 15: додавання експерименту, пошук за дослідником, середнє значення, мінімум і максимум, фільтрацію за діапазоном. Додатково реалізовано консольне меню з валідацією введення, сортування, пошук за кількома критеріями та власні винятки. Історія розробки зафіксована в Git логічними комітами, а README описує встановлення та запуск. Цей проєкт є основою для наступних лабораторних робіт.